# 实验三：从 start_kernel 到 init——跟踪 Linux 内核的启动过程

**真实姓名：** 姚翔
**学号：** 20262822
**实验日期：** 2026 年 10 月 17 日
**报告整理日期：** 2026 年 10 月 18 日
**实验环境：** 32 位 Ubuntu、Linux 3.18.6、QEMU、GDB、MenuOS。

**原创声明：** 本报告的分析与整理署名姚翔，原创作品转载请注明出处；课程要求与内核机制参考《Linux 内核分析》MOOC 及 Linux 上游源码，相关代码保留原作者权利。
**课程出处：** 《Linux 内核分析》MOOC。

## 实验目标与过程分析

### 1. 实验环境与操作步骤

本次实验在蓝桥云课实验楼的 32 位 Ubuntu 环境中完成，使用课程提供的 Linux 3.18.6、QEMU、GDB 和 `rootfs.img`。先进入实验目录，启动内核并加载根文件系统：

```
cd ~/LinuxKernel
qemu -kernel linux-3.18.6/arch/x86/boot/bzImage -initrd rootfs.img \
     -nographic -append "console=ttyS0"
```

`-kernel` 指定可启动内核镜像，`-initrd` 把初始根文件系统映像交给内核。实验指导采用图形窗口，本次操作增加 `-nographic` 和 `console=ttyS0`，让内核输出与 MenuOS 交互出现在终端中，便于观察启动过程。

后续阅读与调试主要涉及以下文件：

```
LinuxKernel/
├── linux-3.18.6/
│   ├── arch/x86/boot/bzImage       — QEMU 加载的启动镜像
│   ├── vmlinux                     — GDB 加载的 ELF 文件与调试符号
│   ├── init/main.c                 — start_kernel、rest_init、kernel_init
│   ├── init/init_task.c            — 初始任务与初始线程栈的静态定义
│   └── kernel/sched/idle.c         — idle 执行路径
├── rootfs.img                      — 包含用户态 /init 的初始根文件系统
└── menu/                           — 课程 MenuOS 程序相关材料
```

`bzImage` 与 `vmlinux` 来自同一次内核构建，分别用于启动与符号调试；修改内核后，应重新编译并同时更新二者。下面先观察内核是否进入用户态程序，再用断点回答“启动过程中谁在执行、在哪里创建新任务”。

### 2. 运行结果：内核启动后为什么会出现 MenuOS

内核完成必要初始化后，继续装载初始根文件系统中的用户程序。终端出现 `MenuOS>>`，说明本次启动已经进入可交互的 MenuOS，而不只是停留在打印内核日志的阶段。

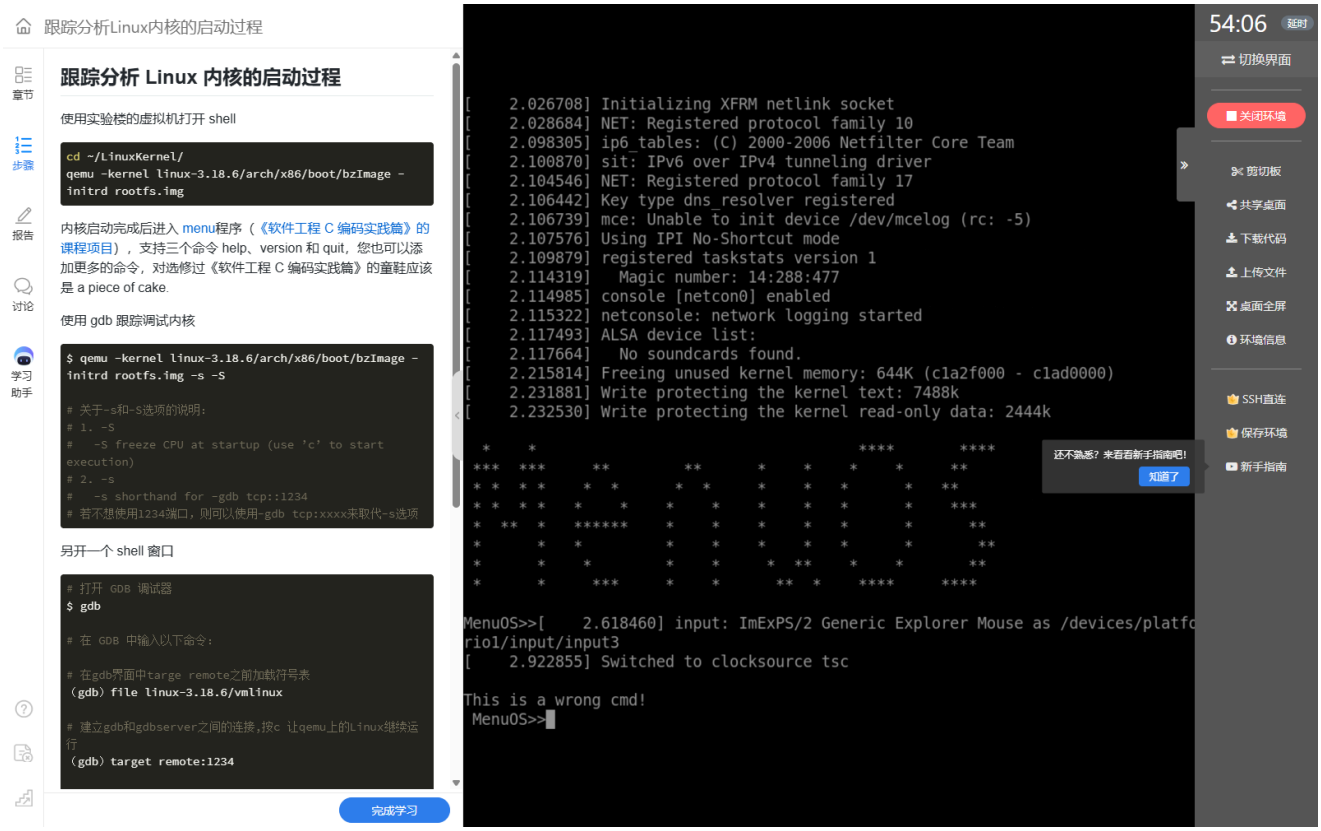



**图1：** 启动日志尾部与 MenuOS>> 提示符；图中的错误命令提示来自菜单输入，不表示内核启动失败。

输入 help 后，菜单列出 help、version、quit 等命令。这里要区分内核与用户程序：命令表属于 MenuOS，内核负责为它提供运行环境，并不直接把 help 当成自己的启动函数执行。

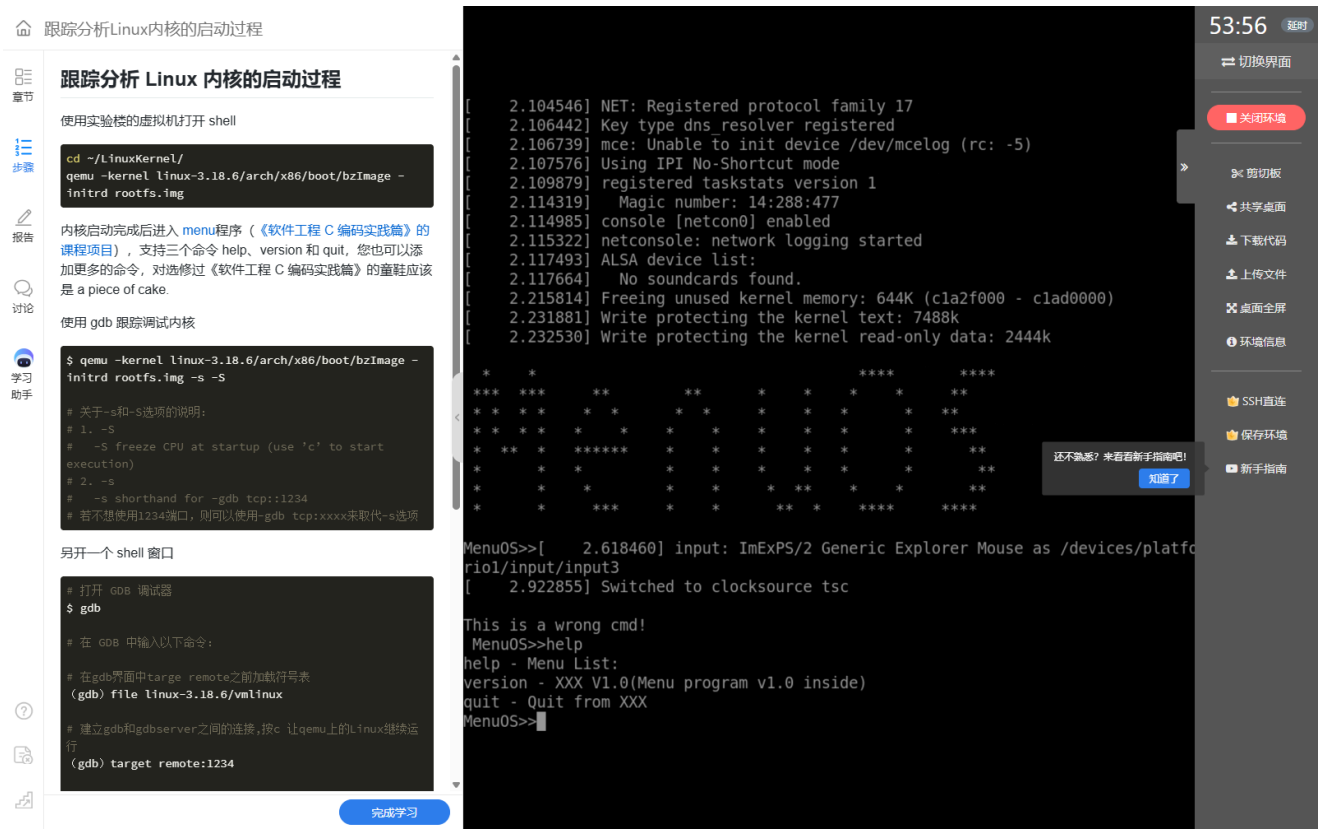



**图2：** help 显示菜单命令及说明，确认用户程序能够接收输入并处理命令。

为了从输出中找出启动阶段，我查看了关键日志。`Linux version 3.18.6` 用于确认内核版本，`Kernel command line: console=ttyS0` 对应启动参数，PID 哈希表、内存和处理器初始化信息说明基础子系统依次投入工作；`Freeing unused kernel memory: 644K` 表示初始化专用内存被回收。

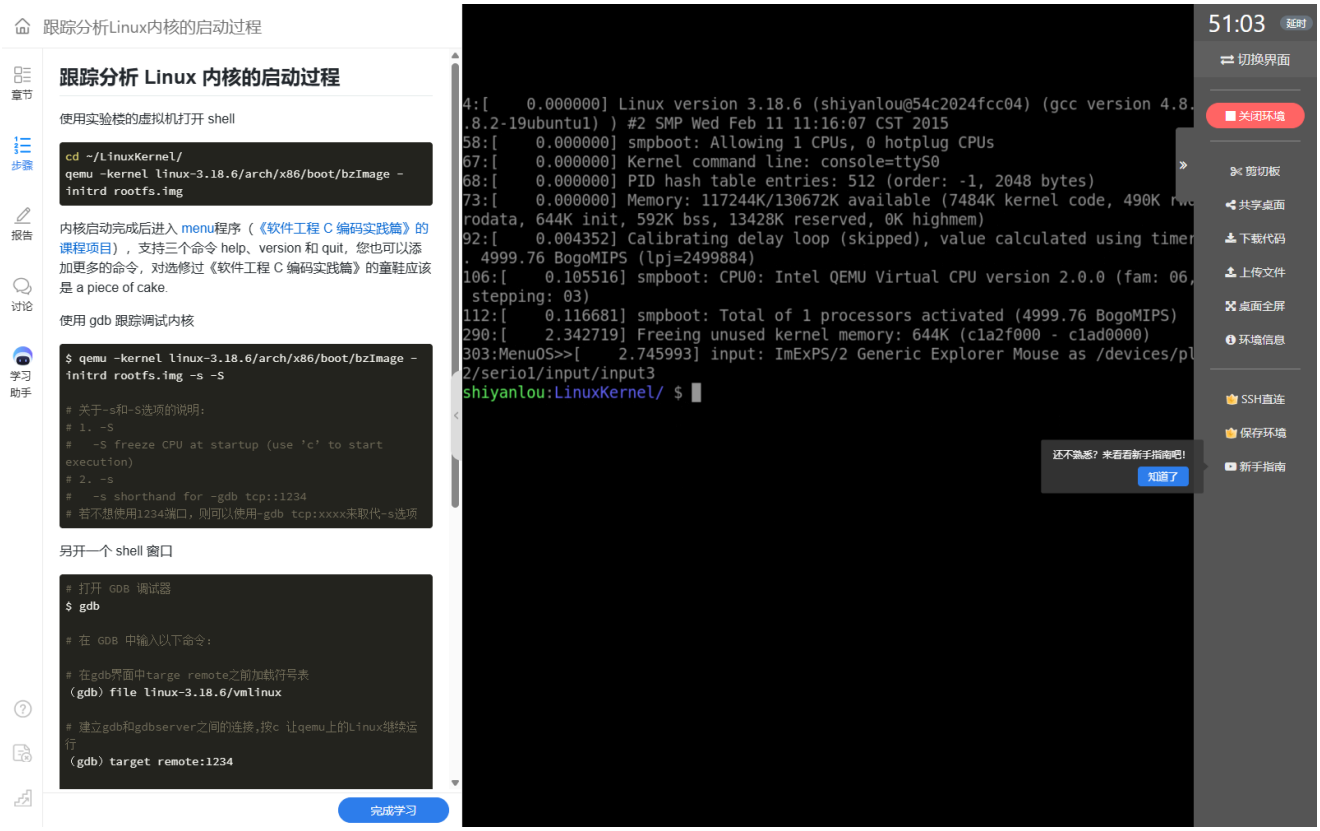



**图3：** 日志记录了版本、单 CPU 配置、PID 哈希表、内存、延时校准和初始化内存回收等节点。日志能说明阶段，具体执行关系还需要用断点和源码确认。

## 3. 建立调试连接：先暂停启动，再加载符号

为了在内核进入 `start_kernel` 前取得控制权，启动另一轮带调试选项的 QEMU：

```
cd ~/LinuxKernel
qemu -kernel linux-3.18.6/arch/x86/boot/bzImage -initrd rootfs.img \
     -display none -serial file:boot2.log -s -S &
```

`-s` 开启默认 TCP 1234 端口的 GDB 调试接口，`-S` 暂停客户机 CPU，二者大小写不能互换。此处串口输出写入 `boot2.log`，所以观察控制台与前面的交互启动使用不同的输出方式；调试连接正常后，用 GDB 的 `continue` 才让客户机继续运行。QEMU 官方调试文档说明了这两个选项的区别。

另开终端，在相同目录加载与启动镜像配套的符号：

```
file linux-3.18.6/vmlinux
target remote:1234
```

GDB 的 `file` 命令读取 ELF 和符号信息，并不代替 QEMU 启动内核。出现 `Reading symbols ... done` 后，调试器才能把地址与函数名、源码位置联系起来。

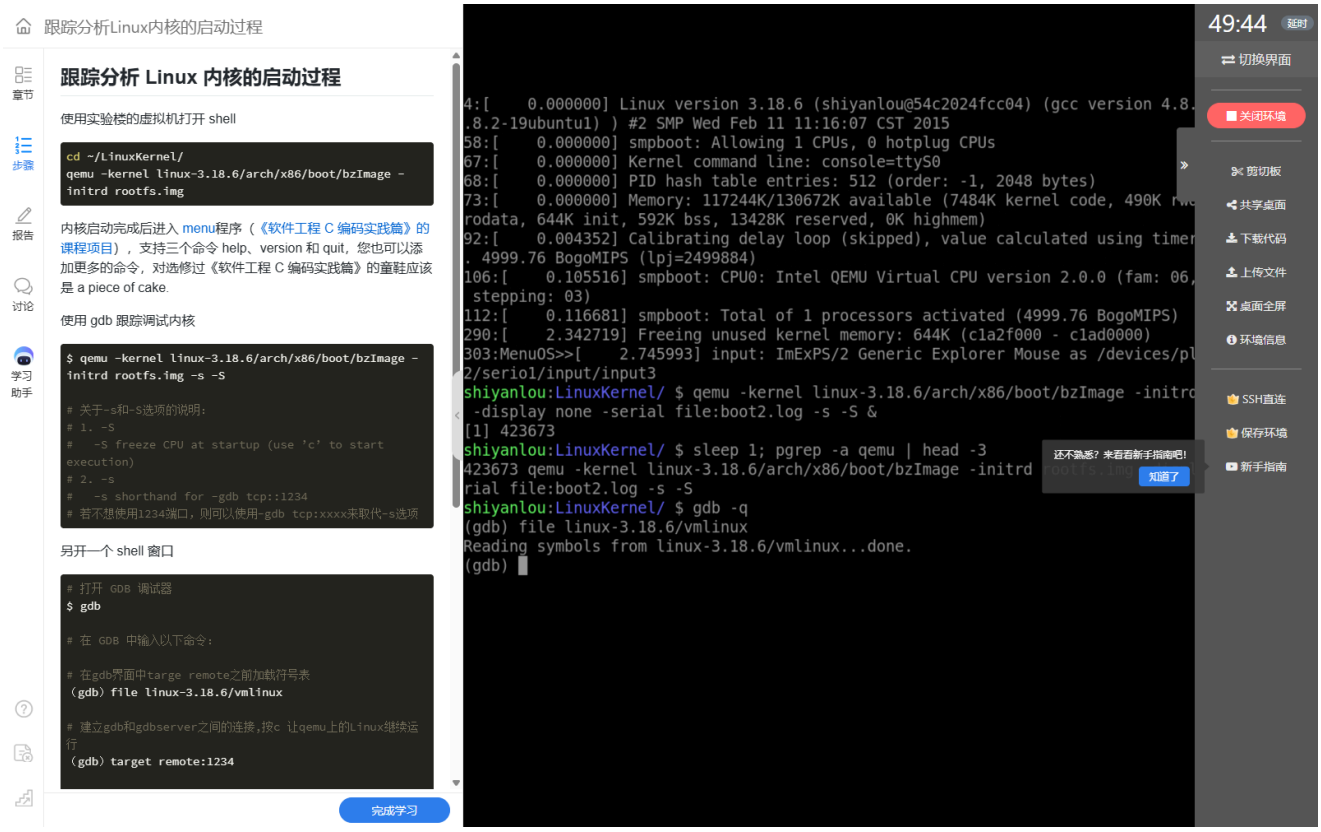



**图4：** `file linux-3.18.6/vmlinux` 成功读取符号。

`target remote:1234` 建立连接后，GDB 显示 `0x0000fff0 in ?? ()`。这是 CPU 复位阶段的地址表现，尚未进入带有 Linux C 语言符号的执行位置，不能把它写成“内核第一条指令”，也不能仅凭 `??` 判断符号文件加载失败。

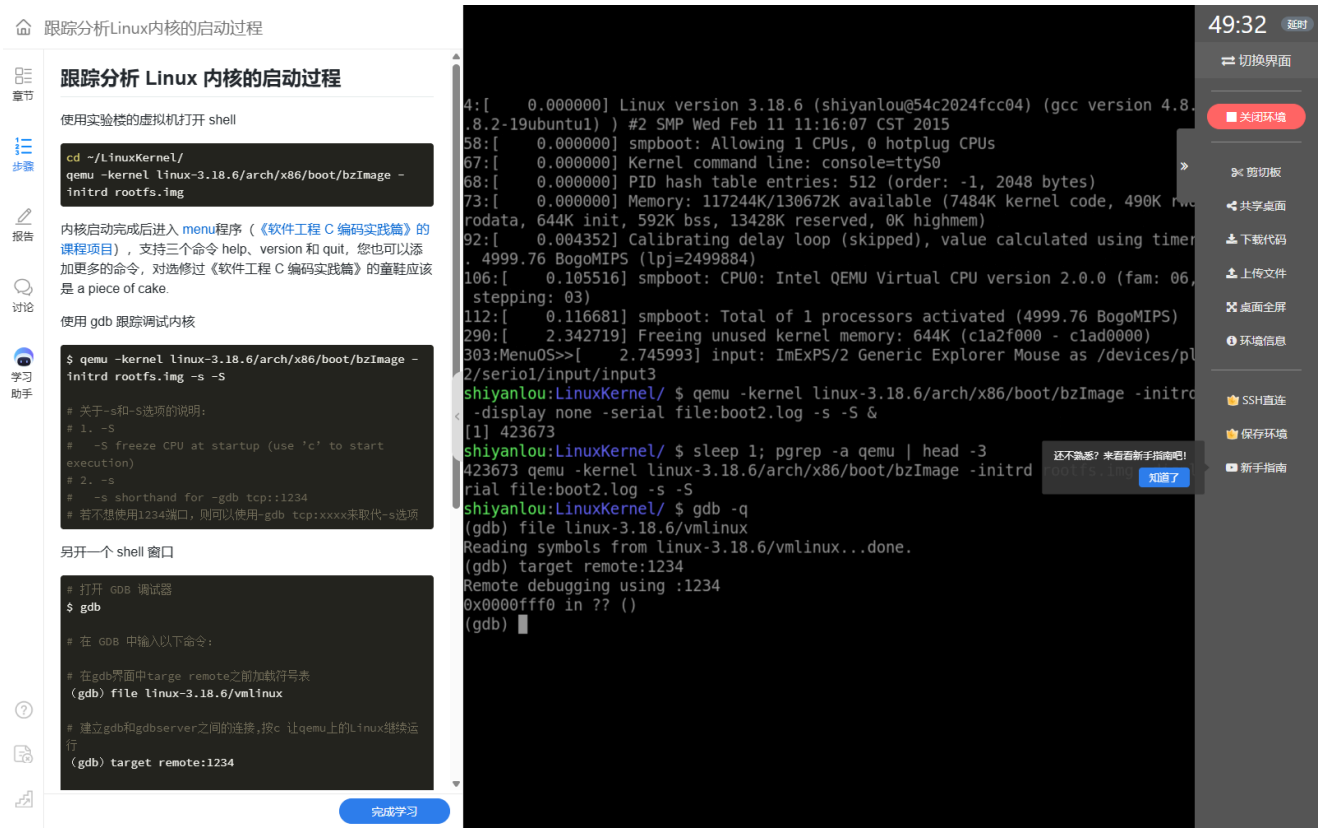



**图5：** 远程连接成功，客户机停在早期复位位置，随后可设置内核断点。

## 4. 跟踪 start_kernel：从地址定位到初始化主线

```
break start_kernel
continue
list
bt
```

先设置符号断点再继续执行，避免内核已经越过目标。截图中断点命中 `start_kernel()`，位置为课程源码 `init/main.c:501`，表明启动镜像和调试符号已经能够配合工作。

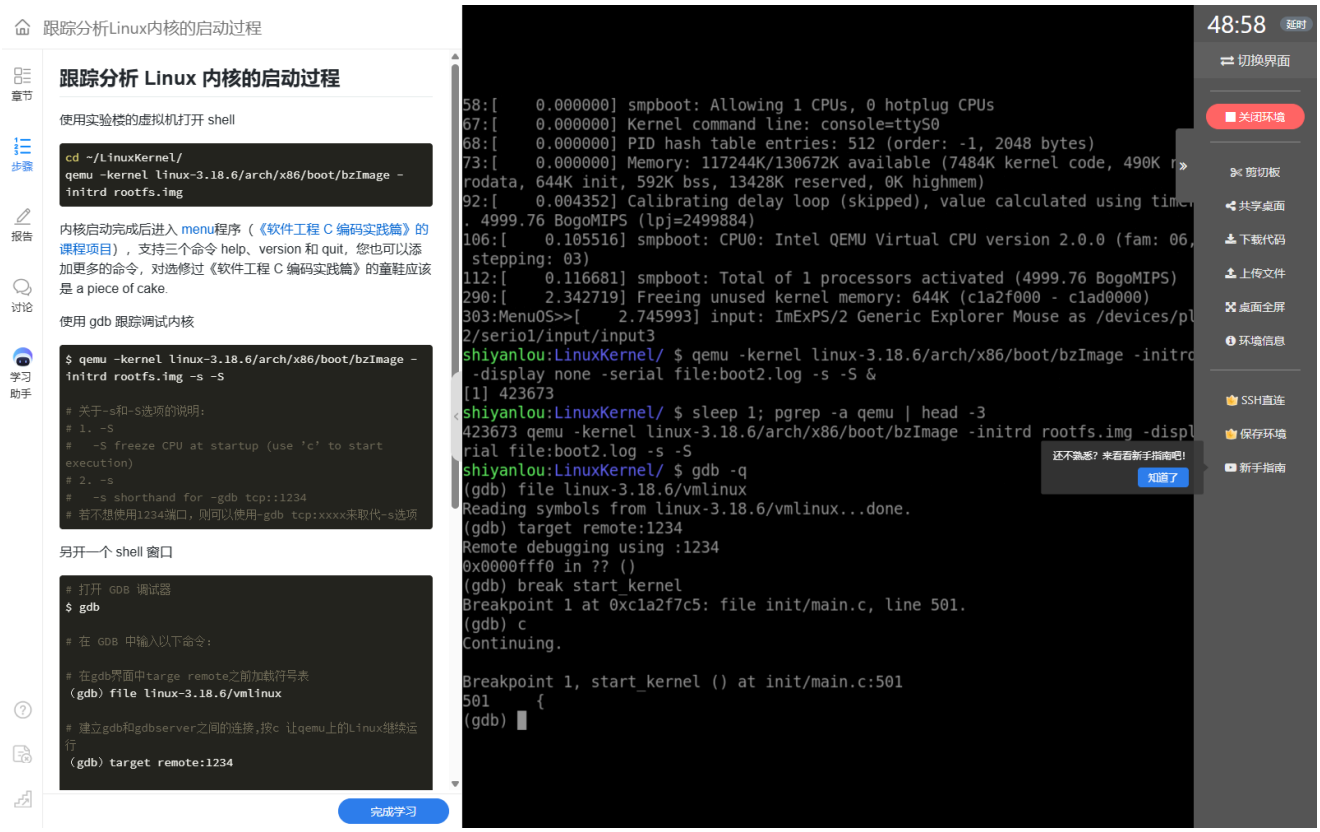



**图6：** `Breakpoint 1, start_kernel () at init/main.c:501`，进入通用内核初始化入口。

`list` 给出函数入口附近的代码，`bt` 显示它的上层为 `i386_start_kernel()`。后者位于 `arch/x86/kernel/head32.c`，本身也是 C 函数；再往前才需要继续分析体系结构相关的汇编启动准备。因此，`start_kernel` 是通用初始化的 C 入口，但不能据此说它一定是整个启动过程中执行的第一个 C 函数。

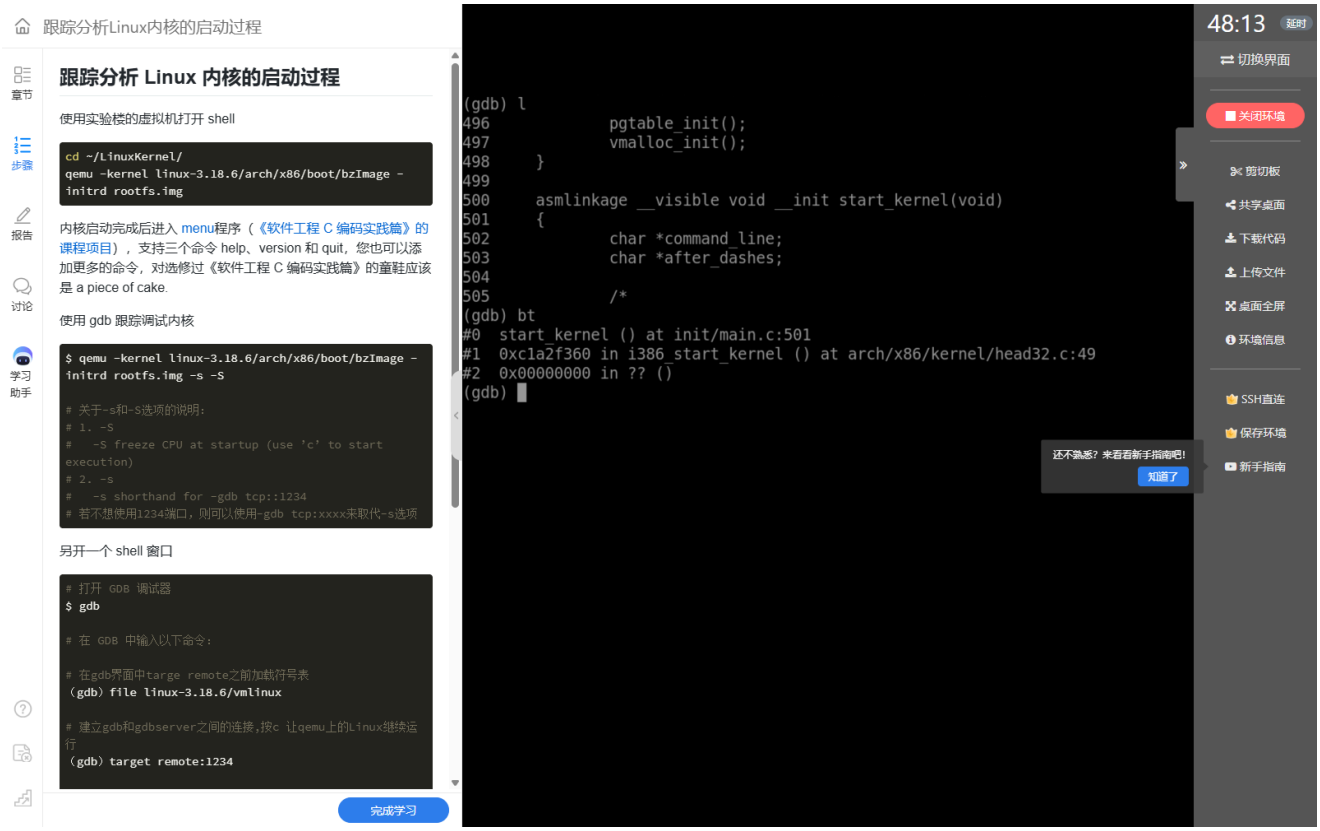



**图7：** 源码入口与 `start_kernel` ← `i386_start_kernel` 的调用关系。最底部未知栈帧不作为额外的函数调用证据。

### （1）为什么 start_kernel 不能当成普通 main 来理解

普通程序的 `main` 执行前，操作系统已提供进程、虚拟内存和运行库环境；`start_kernel` 则是在前期引导建立最低运行条件后，继续把内核自己的服务组织起来。函数运行时已有初始任务 `init_task` 和初始内核栈，并非等执行到 `rest_init` 才第一次出现任务描述符。

### （2）按依赖关系分析初始化过程

以下按 Linux 3.18.6 的实现归纳主要阶段，省略配置分支及辅助检查，不把简化顺序当作可替换原函数的完整源码。课程源码的行号以 GDB 实测为准，上游同版本代码用于对照函数及依赖关系。

| 阶段 | 主要函数或操作 | 解决的问题与先后关系 |
| --- | --- | --- |
| 初始任务状态 | `set_task_stack_end_magic` | 为初始任务建立栈末端检查标记 |
| 早期中断与启动 CPU | `local_irq_disable`、`boot_cpu_init` | 先控制早期中断，再准备启动 CPU 状态，避免中断设施未齐备时过早响应普通可屏蔽中断 |
| 体系结构与参数 | `setup_arch`、`setup_command_line`、`parse_early_param` | 处理架构相关资源及启动参数；串口 console 参数也由启动路径处理 |
| 内存基础设施 | `build_all_zonelists`、`page_alloc_init`、`mm_init` | 准备页分配及更高层分配器，为后续对象创建提供内存 |
| 异常与调度基础 | `trap_init`、`sched_init` | 建立异常处理与调度基础；调度器应先于正常时钟中断投入使用 |

| 阶段 | 主要函数或操作 | 解决的问题与先后关系 |
| --- | --- | --- |
| 中断及时间体系 | `early_irq_init`、`init_IRQ`、`tick_init`、`init_timers`、`timekeeping_init` | 准备 IRQ、时钟和定时器，随后才允许正常中断处理 |
| 开中断与控制台 | `local_irq_enable`、`console_init` | 在必要设施已就绪后允许中断，并组织控制台输出 |
| 进程及文件系统基础 | `pidmap_init`、`fork_init`、`proc_caches_init`、`vfs_caches_init`、`proc_root_init` | 为进程标识、进程创建及文件访问建立基础结构 |
| 转入启动收尾 | `rest_init` | 创建后续任务，让初始执行流进入 idle 路径 |

理解这段代码的重点是依赖关系：没有内存分配基础，后面的任务对象难以创建；没有异常、中断和调度设施，就不应让周期性中断任意打断启动过程。“先 `sched_init`，再完成时钟与 IRQ 初始化，之后开中断”比只记一长串函数名更能解释其设计。

## 5. 跟踪 rest_init：启动路径从这里分成不同任务

```
break rest_init
continue
```

断点命中 `rest_init()`，课程源码位置为 `init/main.c:394`。此时通用初始化已经走到收尾，接下来需要创建承担后续初始化工作的任务，而初始任务保留为 idle。

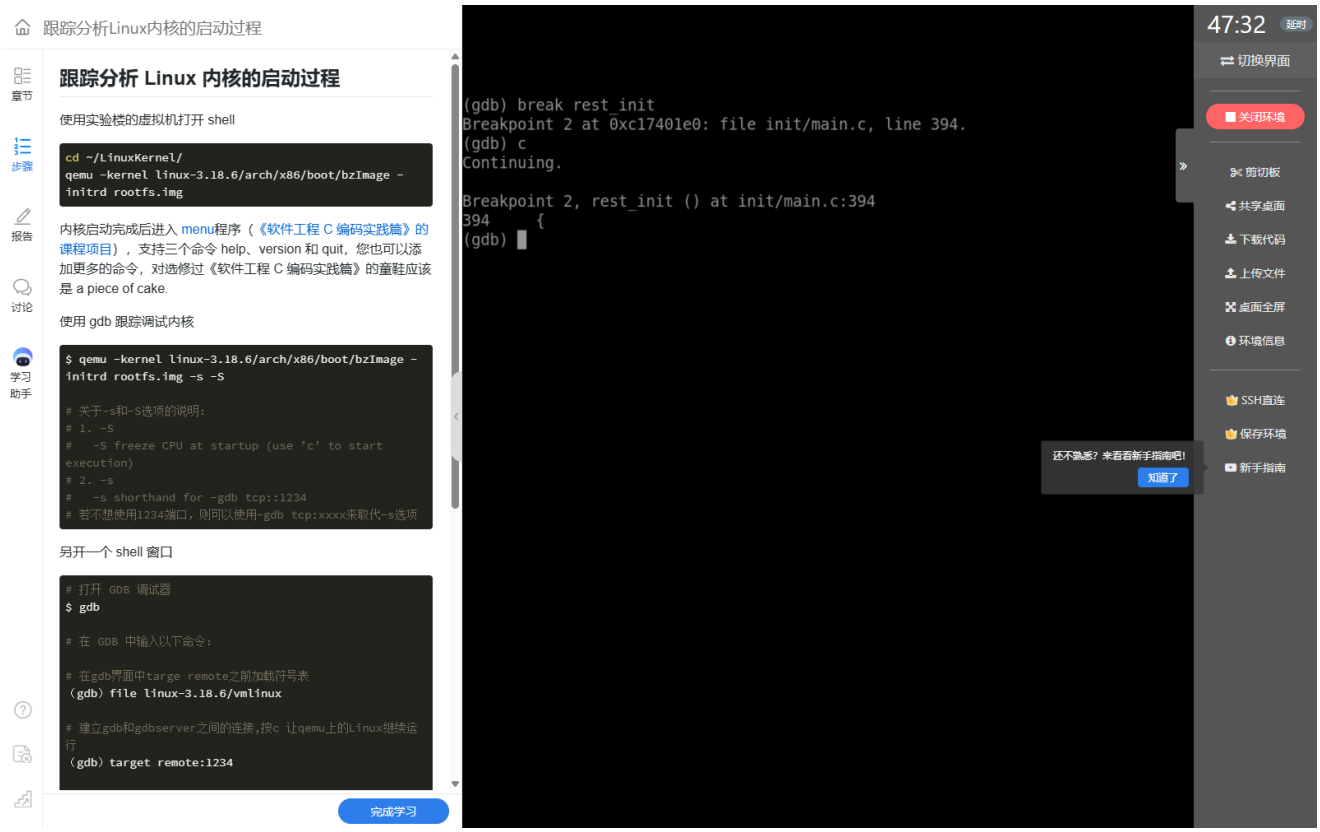



**图8：** 命中 `rest_init`，即分析 0、1、2 号任务分工的重要位置。

下面摘出两个创建动作及启动任务的后续去向；中间的同步与其他准备代码未列出：

```
kernel_thread(kernel_init, NULL, CLONE_FS);
pid = kernel_thread(kthreadd, NULL, CLONE_FS | CLONE_FILES);
/* 创建与同步完成后，启动任务执行一次调度，再进入 idle。 */
schedule_preempt_disabled();
cpu_startup_entry(CPUHP_ONLINE);
```

第一个创建动作先执行，使承担初始化的任务得到 PID 1；第二个动作创建 `kthreadd`，通常为 PID 2。`kernel_init_freeable` 会等待 `kthreadd_done`，与 `rest_init` 中的完成通知配合，避免 PID 1 在内核线程管理设施尚未准备好时继续相关初始化。

这里的创建关系不是普通的直接调用：`kernel_thread(kernel_init, ...)` 创建可调度的新任务，`kernel_init` 在新任务取得 CPU 时运行；原任务则沿自己的内核栈进入 `cpu_startup_entry`。源码对照见 Linux 3.18.6 的 init/main.c。

## 6. 跟踪 kernel_init 与 /init：同一个 PID 的两种执行阶段

### （1）PID 1 先在内核中执行初始化

```
break kernel_init
continue
```

截图命中 `kernel_init(unused=0x0)`，课程源码为 `init/main.c:931`。此时 PID 1 正执行内核初始化函数，还没有因为函数名包含 `init` 就自动变成用户态程序。

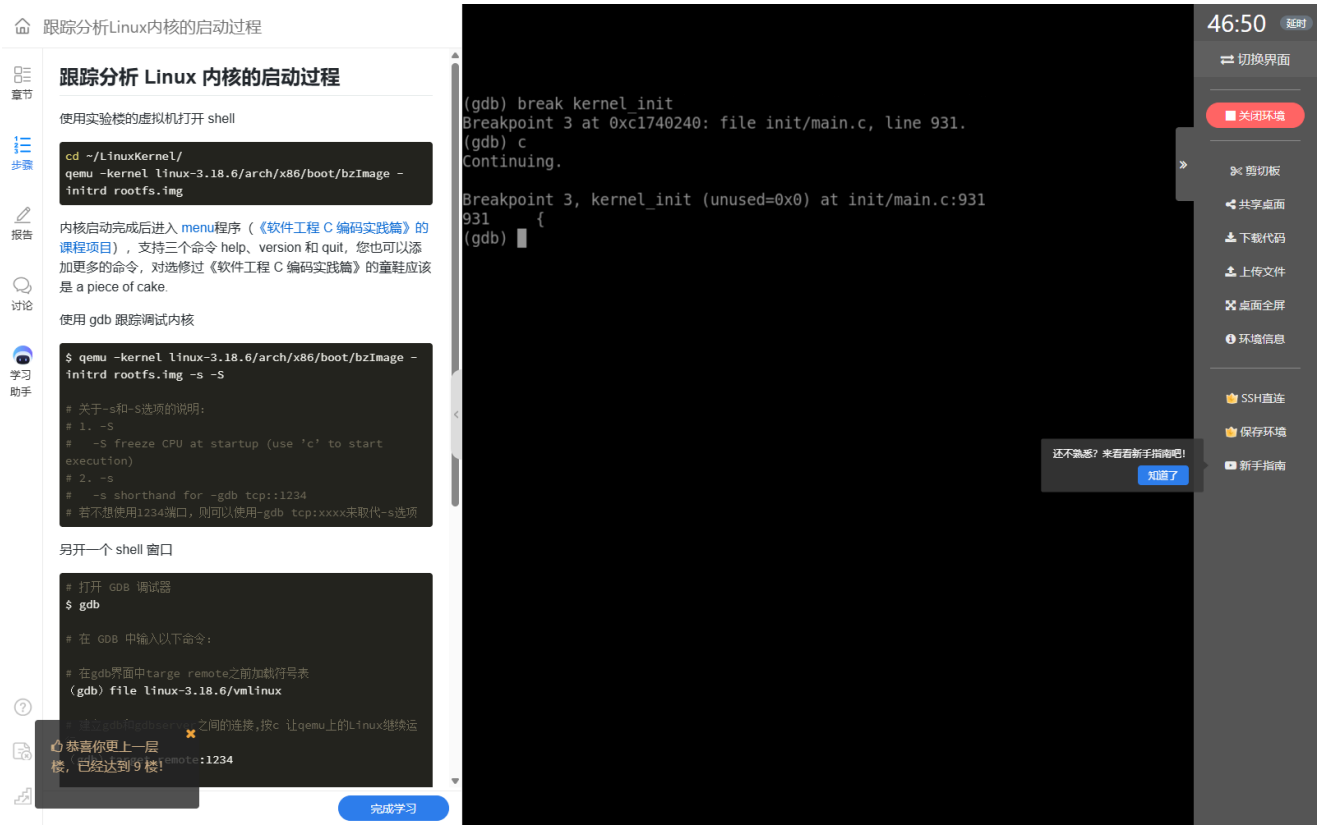



**图9：** `kernel_init` 断点表明新任务开始承担启动收尾工作。

`kernel_init_freeable` 继续处理设备与 initcall 初始化，准备控制台和根文件系统相关条件；随后等待异步初始化完成、回收初始化专用内存，并准备执行用户态初始化程序。若初始根文件系统中存在 `/init`，本版本优先尝试该程序；需要时才继续其他 init 路径。

### （2）run_init_process 的参数确认要装载谁

```
break run_init_process
continue
bt
```

断点命中时参数为 `init_filename="/init"`，对应课程源码 `init/main.c:907`。这比只看到菜单提示符更具体：它定位到内核准备装载哪个用户程序，但该断点仍处于内核中的装载入口。

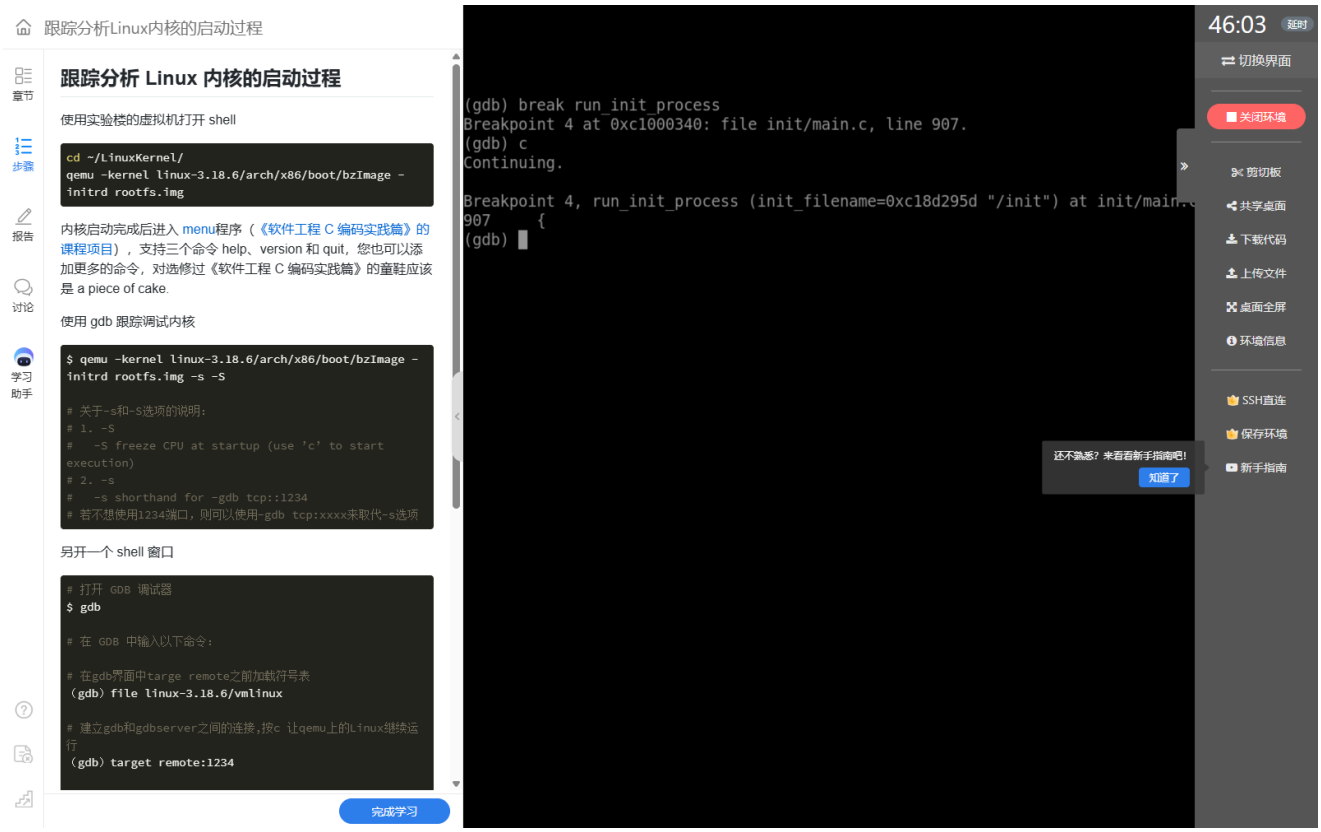



**图10：** `run_init_process` 收到 `/init`，说明本次启动选择了初始根文件系统中的这个程序。

调用栈的前三个可辨认帧依次为 `run_init_process`、`kernel_init`、`ret_from_kernel_thread`。它支持“PID 1 先以新建内核任务的入口运行，随后准备装载用户程序”的解释。更下方出现 `??` 和回溯停止提示，不应把未知帧强行标成 `rest_init` 的普通调用帧。

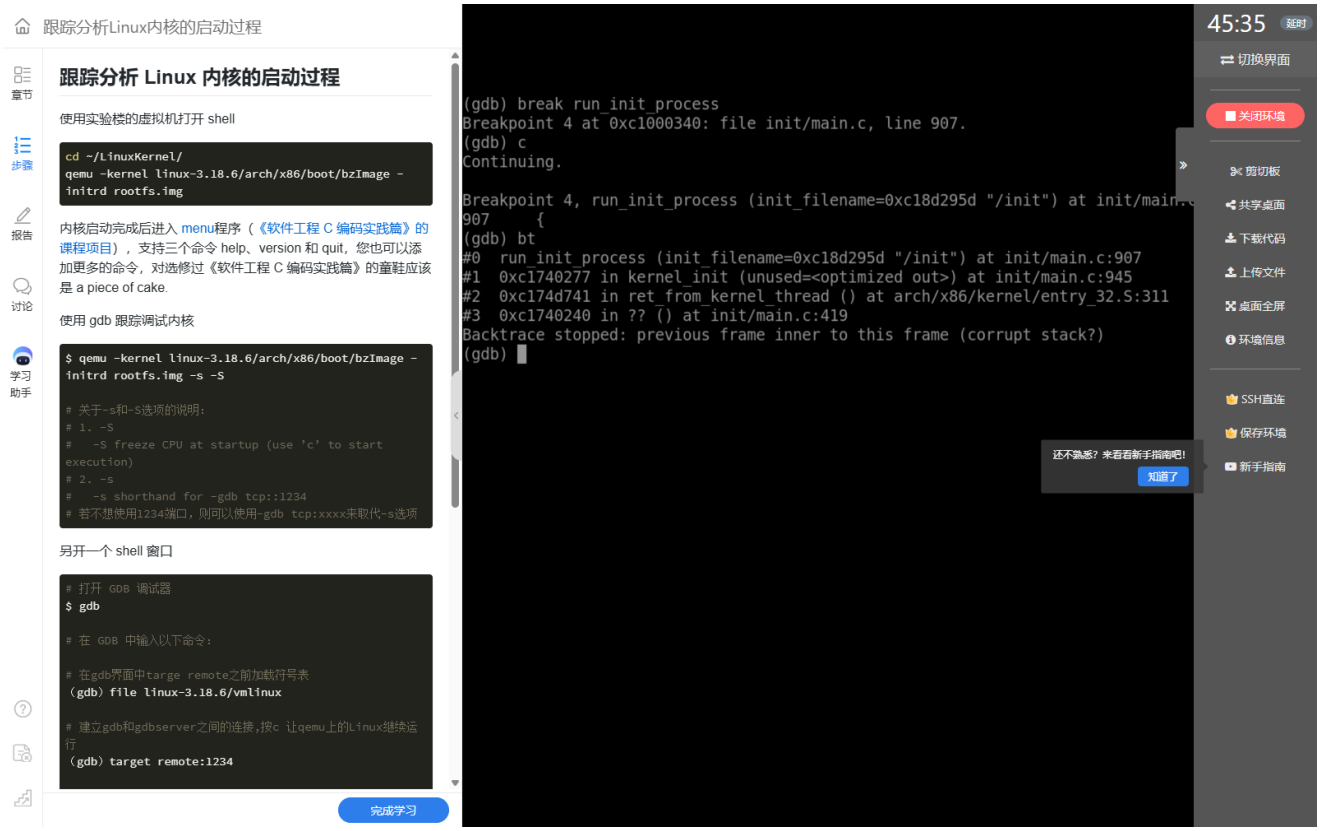



**图11：** 可辨认的调用关系与内核线程启动返回入口；跨启动边界的完整因果关系需要结合任务创建源码理解。

### （3）exec 成功后为什么仍是 1 号进程

`run_init_process` 把程序路径放入启动参数，并进入 `do_execve` 路径。exec 的作用是装载并替换当前任务的用户程序映像，设置新的用户地址空间及返回现场，不是再创建一个 PID。装载成功并经架构相关返回路径进入用户态后，执行的才是 `/init` 的用户代码，本实验表现为 MenuOS。

因此，要分清三个时间点：创建 PID 1；PID 1 执行 `kernel_init`；同一 PID 1 经 exec 执行用户态 `/init`。命中 `run_init_process` 只能直接证明准备执行 `/init`，图1、图2中的用户交互与装载机制共同补全后续过程。Linux 3.18.6 的 exec 实现可用于对照这一区别。

## 7. 跟踪 idle：0 号进程不是最后 fork 出来的

让内核继续执行，在 GDB 中按 `Ctrl+C` 暂停，再运行 `bt`。本次停在 `default_idle`，调用栈向上能找到 `cpu_startup_entry`、`rest_init`、`start_kernel` 和 `i386_start_kernel`。

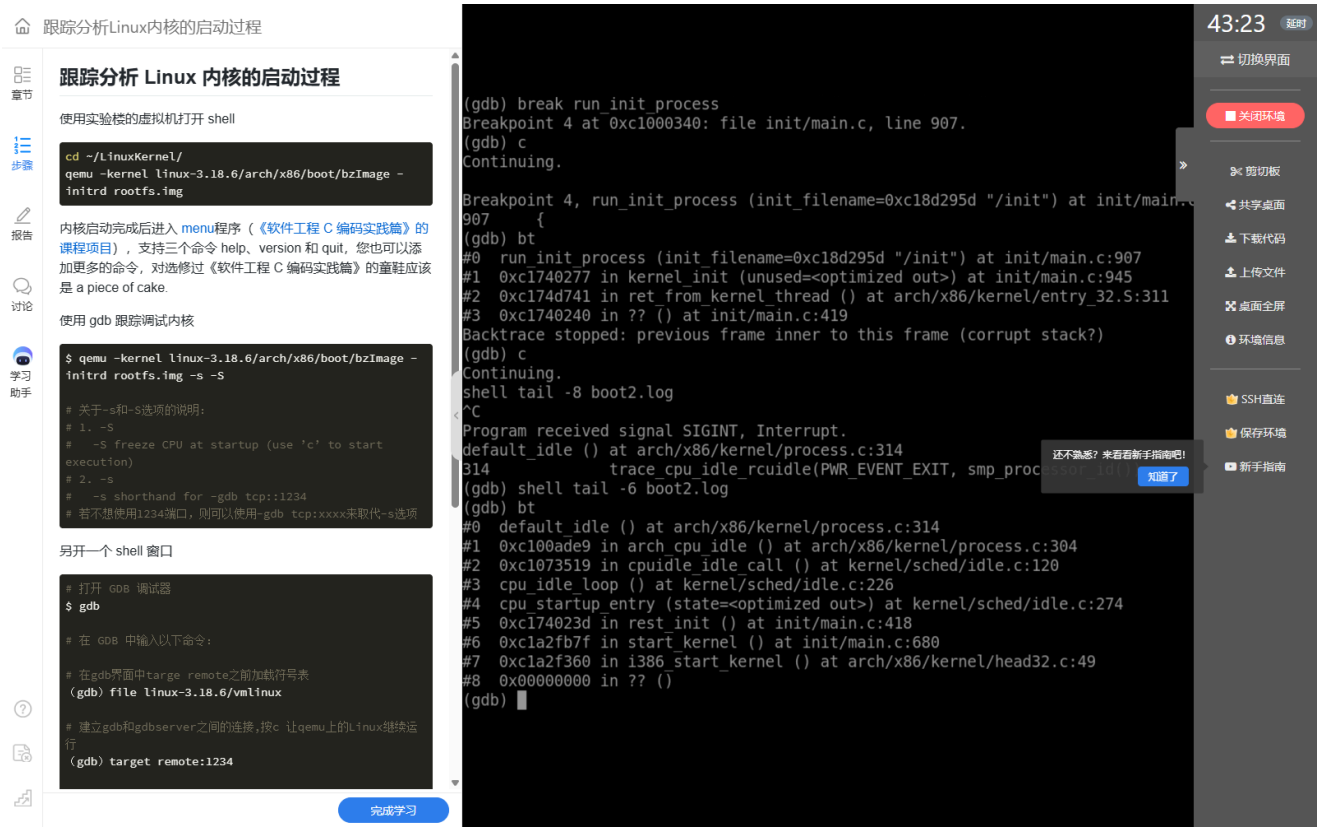



**图12：** 原始启动执行流最终进入 idle；这条栈与 PID 1 的 `kernel_init → run_init_process` 栈属于不同任务路径。

把截图中可辨认的路径按执行方向整理如下：

```
i386_start_kernel
  → start_kernel
  → rest_init
  → cpu_startup_entry
  → cpu_idle_loop
  → cpuidle_idle_call / arch_cpu_idle
  → default_idle
```

初始任务 `init_task` 和初始线程栈在源码中静态定义，引导代码在进入通用初始化前已建立相关执行环境；它不是由 `rest_init` 再 fork 出来的。启动后，原执行流在无普通可运行任务时承担 idle 工作。idle 也不等于始终浪费 CPU 的简单死循环：架构实现可以通过等待中断等机制进入空闲状态，之后响应唤醒和调度需要。参见 初始任务定义与 idle 实现。

## 8. 用两条执行路径总结 0、1、2 号进程

```
静态初始任务 / 启动 CPU 执行流（PID 0）
  i386_start_kernel → start_kernel → rest_init
                                        |
                                        ├─ 创建 PID 1，入口 kernel_init
                                        |     → 后续初始化及装载准备
                                        |     → run_init_process("/init")
                                        |     → exec 成功后进入用户态 MenuOS
                                        |
                                        ├─ 创建 PID 2，入口 kthreadd
                                        |     → 管理常规内核线程创建请求
                                        |
                                        └─ 原 PID 0 执行调度并进入 idle
                                              → cpu_startup_entry → 空闲循环
```

上图中的“创建”表示新任务的产生，不表示当前内核栈直接调用新任务的完整执行流程。PID 1 和 PID 2 获得运行机会后才沿各自的栈执行，PID 0 的 idle 栈不会变成 PID 1 的用户栈。

| 任务 | 来源 | 主要职责 | 与截图的对应关系 |
| --- | --- | --- | --- |
| PID 0：启动 CPU 的 idle/swapper | 静态初始任务及引导执行流 | 早期初始化，随后在该 CPU 无其他可运行任务时进入空闲路径 | 图7与图12串起通用初始化和 idle |
| PID 1：init | `rest_init` 创建，入口为 `kernel_init` | 完成后续初始化，再装载用户态 init 程序 | 图9—图11定位内核入口与 `/init` 装载准备；图1、图2显示用户交互 |
| PID 2：kthreadd | `rest_init` 创建 | 承担常规内核线程创建请求 | 通过 `rest_init` 源码与初始化同步关系分析 |

“0 号进程先存在、1 号进程被创建、1 号进程再装载用户程序”是本实验最重要的区分。`kthreadd` 也有明确分工，但不能笼统地说所有内核线程都由它创建，因为 PID 1、PID 2 的早期创建就是启动路径直接完成的。

## 9. quit 引发 panic：init 为什么不能像普通程序一样退出

本次记录中，执行菜单退出操作后出现 `Attempted to kill init!`，同时能看到 `PID: 1 Comm: init` 和退出相关调用。输出还伴随保护异常与递归故障信息，所以不能把这一屏全部视为正常退出信息。

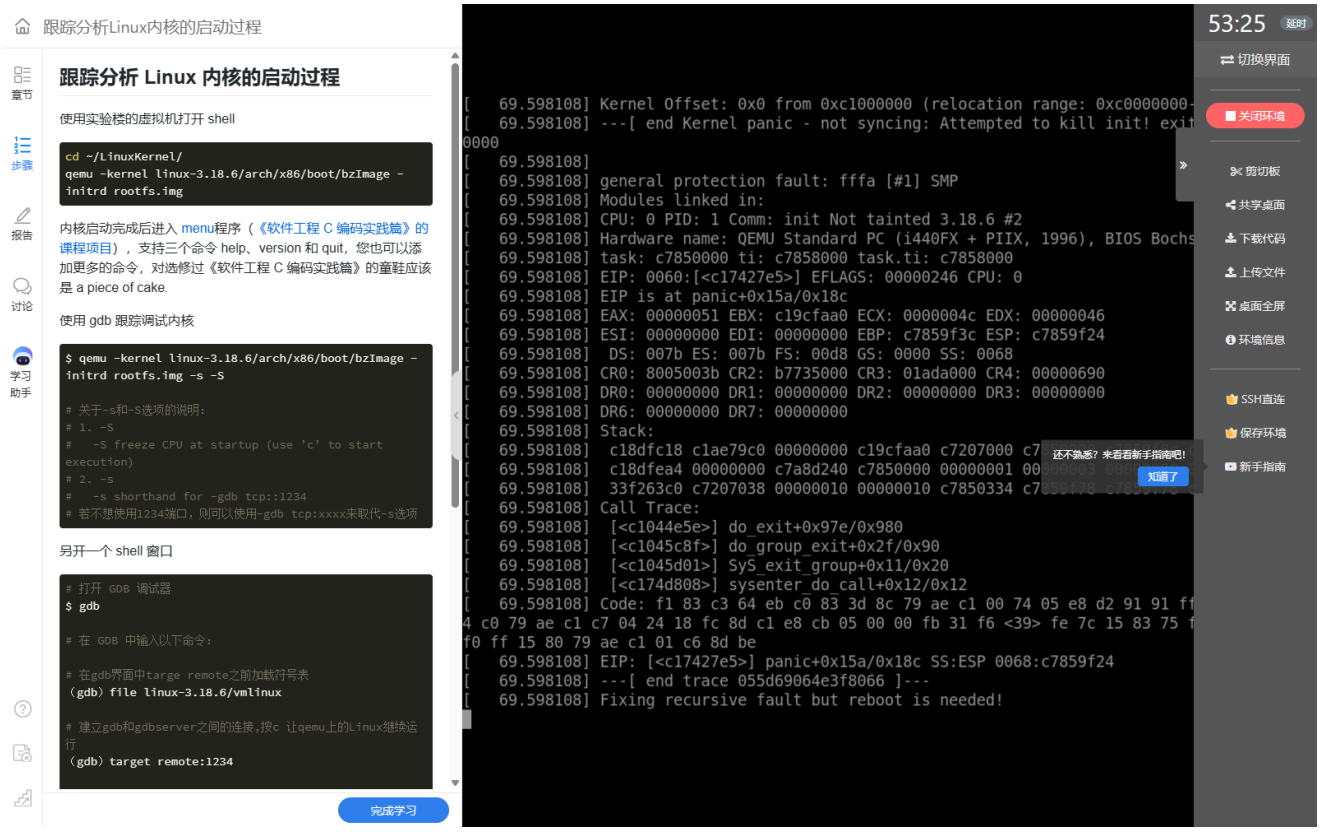



**图13：** panic 信息与 PID 1 标识，显示当前初始化程序退出会破坏系统的基本用户态运行条件。

结合 Linux 3.18.6 的退出处理源码，初始 PID 命名空间的 init 具有特殊地位，失去该任务会触发内核保护。该现象与本次 `/init` 身份一致，也提醒我：判断启动成功应结合已经出现的菜单交互与断点，而不应被最后一次退出后的 panic 反向误导成“内核从未成功启动”。

## 10. 遇到的问题、分析与解决方案

| 问题或疑问 | 原因分析 | 解决方法与核验依据 |
| --- | --- | --- |
| QEMU 加 `-S` 后没有继续输出，是不是卡死了? | CPU 被主动暂停，等待调试器 | 按 `file → target remote → break → continue` 操作，图4—图6能接续验证 |
| 为什么 QEMU 用 bzImage，GDB 用 vmlinux? | 启动格式与符号调试用途不同 | 保持二者来自同次构建；图4的符号加载和图6的源码断点相互印证 |
| 连接后看到 `0x0000fff0 in ??` 怎么办? | 连接时仍在早期复位位置 | 不把它当 Linux C 入口，设置 `start_kernel` 断点后继续；图5、图6展示了两个不同阶段 |
| 为什么只看日志还不能说明谁创建 init? | 日志反映阶段，未直接给出任务创建语义 | 对照 `rest_init` 的创建动作，再看 `kernel_init` 与 `/init` 断点 |
| `kernel_init` 是否就是用户态 `/init`? | 函数名相似，但程序映像和执行阶段不同 | 分开记录创建、内核初始化、exec 装载三个环节；图9—图11验证其联系 |
| 在 `run_init_process` 断点就进入用户态了吗? | 函数正处于内核装载入口 | 把断点写成“准备装载”，把 exec 后用户现场恢复与菜单交互作为后续环节 |
| `bt` 出现未知帧或 `corrupt stack`? 如何解释? | 优化、线程入口和早期启动现场会影响回溯 | 采用可辨认帧，结合创建源码与另一任务的 idle 栈交叉判断，不扩写未知帧 |

| 问题或疑问 | 原因分析 | 解决方法与核验依据 |
| --- | --- | --- |
| 0 号进程是不是 `rest_init` 创建的? | 混淆了静态初始任务与后续动态创建 | 查 `init_task` 定义，并用图12追踪原启动执行流进入 idle |
| 上游代码行号为什么与实验不同? | 课程内核树可能包含教学修改 | 操作以函数名断点为主，实测行号按截图记录，不照搬上游行号 |
| 为什么退出菜单后内核 panic? | 用户态初始化程序是特殊的 PID 1 | 对照图13与退出保护源码，区分已完成的启动与退出造成的新状态 |

### 11. 我对 Linux 内核启动过程的理解

这次实验让我把“内核启动”从一屏快速滚动的文字，理解为建立运行条件、创建后续任务、装载用户程序的过程。`start_kernel` 主要把内存、异常、中断、调度和文件访问等基础设施准备好；`rest_init` 则把工作分派到不同任务，并保留原始任务承担 idle。

最容易混淆的是“调用函数”和“创建任务”。`rest_init` 创建 PID 1，不表示它在原栈中一路直接调用到 MenuOS；PID 1 有自己的执行路径，先运行 `kernel_init`，再通过 exec 装载 `/init`。exec 更换程序映像，但 PID 1 的身份延续。与此同时，PID 0 在自己的内核栈上进入 idle，系统可以在这些任务间调度。

因此，我认为启动的关键成果是让系统能够从初始化阶段转入以任务调度为基础的持续运行：有可用的内核服务，有用户态初始化程序，也有无普通任务可运行时的 idle 路径。断点、参数、可靠的调用栈和控制台交互必须相互印证，才能准确描述这一过程。

## 参考资料


1. 孟宁、娄嘉鹏：《庖丁解牛 Linux 操作系统分析》，第 4 章“深入理解系统调用”，112—153 页；第 3 章启动与调试内容用于联系实验。
2. 《Linux 内核分析》MOOC 课程，课程实验三“跟踪分析 Linux 内核的启动过程”。
3. 作者提供的课程与教材配套资料。
4. Linux 3.18.6：init/main.c、初始任务、idle 路径、exec 实现、退出处理。
5. QEMU 官方文档：GDB 调试。
6. Linux man-pages：syscall(2)，用于核对架构 ABI；vDSO(7)，用于理解用户态快速路径。
7. Linux 5.4：x86 系统调用入口公共代码，用于对照教材的保存现场、分派及返回前工作。